Transactions Dispute Portal – Core Architecture Overview

Core System Architecture

The Dispute Portal is designed as a **distributed financial platform** with a clear separation between the customer-facing application, the central dispute API, and the backend financial services.

The main architecture is:

Vue 3 Portal
→ .NET 10 Identity and Dispute API
→ Financial Services
→ SQL Server / PostgreSQL

The frontend is built with **Vue 3** and communicates mainly with one customer-facing **.NET 10 API**. The browser does not communicate directly with all financial services.

The API acts as the main application boundary and is responsible for:

- authentication and sessions;
- users, roles and permissions;
- disputes and dispute status;
- evidence;
- support conversations;
- notifications;
- financial projections;
- policy management;
- administration;
- monitoring and audit.

Behind the API, the financial platform is separated into ten backend services:

- Configuration
- Accounts
- Ledger
- Transactions
- Payments
- Notifications

- Audit
- Reporting
- Scheduling
- Reversal Review / Debt

Each service is an independent process with its own responsibility.

Service Ownership

The architecture follows a strong **service ownership** model.

The API owns identity and dispute-related information.

The Accounts service owns account identity and account projections.

The Transactions service owns financial workflow state.

The Payments service owns payment instructions and settlement intent.

The Ledger service owns authoritative financial postings.

The Configuration service owns financial configuration such as countries, currencies, limits and fees.

Other services own reporting, notifications, audit and scheduled financial work.

This keeps financial responsibilities separated instead of placing the whole system into one application or one database.

Database Architecture

The main API uses **SQL Server by default**, while the financial services use separate **PostgreSQL databases**.

Each financial service owns its own database.

There are no cross-database foreign keys and no distributed SQL transactions between services.

Inside the API database, the design uses:

- BIGINT internal primary keys;
- numeric foreign keys;
- GUID public identifiers;
- unique constraints;
- database-level integrity rules.

Public GUIDs are exposed through the API while numeric IDs are used internally for efficient database relationships.

The main API schemas include:

Identity – users, profiles, roles, permissions and sessions.

Accounts – account projections and bank bindings.

Transactions – original financial transaction information.

Disputes – disputes, status history, evidence, refunds and financial commands.

Support – conversations, messages and assignment information.

Notifications – notification and delivery state.

Audit – audit events and actor history.

Configuration – versioned financial and dispute policies.

Monitoring – aggregated application metrics.

Financial Architecture

The financial processing flow is approximately:

Payments
→ Transactions
→ Ledger
→ Accounts
→ Reporting / Audit / Notifications
→ API
→ Vue Portal

The **Ledger service** is the authoritative monetary boundary.

It stores:

- double-entry journals;
- financial positions;
- reservations;
- reversal links;
- authoritative financial postings.

The API may show balances and financial projections, but those projections are not the authoritative ledger.

Page 2 – Communication, Security and Infrastructure

Event-Driven Architecture

Communication between backend services uses an **event-driven architecture**.

Each service commits its business change together with an **outbox record** in the same database transaction.

A background publisher sends the event to the next service.

The receiving service records an **inbox receipt** together with its local business update.

The communication architecture includes:

- transactional outbox;
- transactional inbox;
- RabbitMQ;
- event IDs;
- correlation IDs;
- signed event envelopes;
- publisher confirmations;
- consumer acknowledgements;
- retries;
- dead-letter queues.

RabbitMQ is used for asynchronous messaging when enabled.

The same outbox/inbox architecture can also use authenticated HTTP service delivery.

The Transactions service also keeps durable workflow state such as:

- Initiated
- Validated
- Processing
- Aborting
- Completed
- Failed

This allows financial operations to continue through multiple services while each service still owns its own database.

Policy Architecture

Financial rules are stored as **immutable policy versions**.

The platform maintains versioned:

- refund policies;
- risk policies;
- routing policies;
- platform configuration.

Publishing or updating a policy creates a new version instead of overwriting an old one.

Each version can contain:

- version number;
- priority;
- start and expiry time;
- author;
- reason;
- publication time;
- previous-version references.

A shared policy resolver determines which version is currently effective.

Authentication and Security

Authentication is controlled by the backend.

The API manages:

- password verification;
- sessions;
- OTP verification;
- remembered devices;
- roles;
- permissions;

- cookies;
- bearer authentication;
- antiforgery protection;
- session revocation.

Infobip is used for SMS, email and WhatsApp delivery, but the portal itself manages OTP generation and verification.

The HTTP pipeline includes:

1. Security headers
2. Correlation handling
3. Serilog logging
4. Exception handling
5. Trusted proxy handling
6. HTTPS and HSTS
7. Routing and CORS
8. Authentication
9. Authorization
10. Rate limiting
11. Antiforgery
12. Controllers and SignalR

TLS policy supports TLS 1.2 and 1.3 and secure database and RabbitMQ connections.

Realtime, Monitoring and Deployment

SignalR is used for realtime notifications between the API and Vue frontend.

Monitoring uses built-in ASP.NET Core instrumentation:

API Request
→ ASP.NET Core Metrics
→ MeterListener
→ Aggregation
→ Dapper
→ Monitoring Database
→ Admin Monitoring API

Serilog is used for structured logging.

The deployment architecture uses Docker Compose.

The key architectural idea is that the portal uses a **service-oriented, event-driven financial architecture with clear service ownership, separate persistence boundaries, a central customer-facing API and durable communication between financial services**.